

Efficient Computation of Jogging Routes^{*}

Andreas Gemsa, Thomas Pajor, Dorothea Wagner, and Tobias Zündorf

Department of Computer Science, Karlsruhe Institute of Technology (KIT)
{gemsa,pajor,dorothea.wagner}@kit.edu, tobias.zuendorf@student.kit.edu

Abstract. We study the problem of computing jogging (running) routes in pedestrian networks: Given source vertex s and length L , it asks for a cycle (containing s) that approximates L while considering niceness criteria such as the surrounding area, shape of the route, and its complexity. Unfortunately, computing such routes is NP-hard, even if the only optimization goal is length. We therefore propose two heuristic solutions: The first incrementally extends the route by joining adjacent faces of the network. The other builds on partial shortest paths and is even able to compute sensible alternative routes. Our experimental study indicates that on realistic inputs we can compute jogging routes of excellent quality fast enough for interactive applications.

1 Introduction

We study the problem of computing *jogging routes* in pedestrian networks. Given a source vertex s (the user's starting point), and a desired length L (in kilometers), the problem asks for a cycle of length (approximately) L that contains the vertex s . A “good” jogging route is, however, not only determined by its length; other criteria are just as important. An ideal route might follow paths through nice areas of the map (e.g., forests, parks, etc.), has rather circular shape, and not too many intersection at which the user is required to turn. A practical algorithm must, therefore, take all of these criteria into account.

Much research focused on efficient methods for the related, but simpler, problem of computing point-to-point (shortest) paths. In fact, a plethora of algorithms exist, many of which are surveyed in [3,9]. They usually employ sophisticated preprocessing to speed up query performance. In contrast, much less practical work exists for computing cycles. Graphs may contain exponentially many (in the number of vertices) cycles, even if they are planar [1]. If the length of the cycles is restricted by L , they can be enumerated in time $O((n+m)(c+1))$, where c is the number of cycles of length at most L [8]. If one is interested in computing cycles with exactly k edges, the problem can be solved in $O(2^k m)$ expected time [10]. Unfortunately, none of these methods seem practical in our scenario. To the best of our knowledge, no efficient algorithms that quickly compute sensible jogging routes exist.

This work introduces the JOGGING PROBLEM. It turns out to be NP-hard, hence, we propose two heuristic approaches. The first, *Greedy Faces* is based

^{*} Partially supported by DFG grant WA 654/16-1.

on building the route by successively joining adjacent faces of the network. The second, *Partial Shortest Paths*, exploits the intuition of constructing equilateral polygons via shortest paths. The latter can be easily parallelized and has the inherent property of providing sensible alternative routes. The result of our algorithms are routes of length within $(1 \pm \varepsilon)L$, but also consider other important criteria that optimize the surrounding area, shape, and route complexity. An experimental study justifies our approaches: Using OpenStreetMap data, we are able to compute jogging routes of excellent quality in under 200 ms time, which is fast enough for interactive applications.

The paper is organized as follows. Section 2 defines variants of the problem and shows NP-hardness. Section 3 introduces our two algorithmic approaches. Section 4 presents experiments, and Section 5 contains concluding remarks.

2 Problems

Before we formally define the considered problems, we need to develop some notation. We model pedestrian networks as *undirected graphs* $G = (V, E)$ with nonnegative integral *edge costs* $\ell: E \rightarrow \mathbb{Z}_{\geq 0}$. Usually, vertices correspond to intersections and edges to walkable segments. Also, we assume that our graphs admit straight-line embeddings, since vertices have associated latitude/longitude coordinates. For simplicity, our graphs are always connected. A *path* P is a sequence of vertices $P = [u_1, \dots, u_k]$ for which $u_i u_{i+1} \in E$ must hold. Note that we sometimes just write u_1 - u_k -path or P_{u_1, u_k} for short. If the first and last vertices coincide, we call P a *cycle*. The *cost* of a path, denoted by $\ell(P)$, is the sum of its edge-costs. A *shortest path* between two vertices u_1 and u_2 is a u_1 - u_2 -path with minimum cost. At some places we require intervals around a value $x \in \mathbb{Z}_{\geq 0}$ with error $\varepsilon \in \mathbb{R}_{\geq 0}$. We define them by $I(x, \varepsilon) = [\lfloor (1 - \varepsilon)x \rfloor, \lceil (1 + \varepsilon)x \rceil]$.

Simple Jogging Problem. The first problem we consider is the SIMPLE JOGGING PROBLEM (SJP): We are given a graph G , source vertex $s \in V$, and a targeted cost $L \in \mathbb{Z}_{\geq 0}$ as input. The goal is to compute a cycle P through s with cost $\ell(P) = L$. In practical scenarios, cost usually represent geographical length. It turns out that SJP is NP-hard by reduction from HAMILTONIAN CYCLE. Note that from this, NP-hardness follows for the respective optimization problem, i. e., finding a cycle that *minimizes* $|\ell(P) - L|$.

If we allow running time in the order of L , one can solve SJP by a dynamic program, similarly as it is known for the SUBSET SUM PROBLEM [5]. The algorithm maintains a boolean matrix $Q: V \times \mathbb{Z}_{\geq 0} \rightarrow \{0, 1\}$ of size $|V| \times L$, which indicates whether a path to vertex u with cost ℓ exists. Initially, Q is set to all-zero, except for the entry $Q(s, 0)$, which is set to 1. It then considers subsequent cost values ℓ in increasing order (beginning at 0). In each step, the algorithm checks for all edges $uv \in E$ if an existing path can be extended to v with cost ℓ . It does so by looking if $Q(u, \ell - \ell(uv))$ is set to 1, updating $Q(v, \ell)$ accordingly. The algorithm stops as soon as ℓ exceeds the input cost L . Then, the requested jogging route exists iff $Q(s, L) = 1$ holds. The running time of the algorithm is $O(L|E|)$, and thus we conclude that the SJP is weakly NP-hard [5].